

Initiation à la méthodologie DevOps, Gitlab CI/CD et conteneurisation Docker

Partie 2 (suite) : Reprendre la main sur sa pipeline ClickFast

Votre pipeline ClickFast est verte. Le badge affiche `passing`, les tests tournent, l'image se build. Beau travail. Mais si on regarde ce qu'il y a vraiment dans le fichier YAML, une bonne partie vient du bouton "New Workflow" de GitHub : un stage unique, pensé pour n'importe quel projet Docker, pas pour le vôtre.

Qu'est-ce qui, dans ce fichier généré, correspond réellement à ce qu'on a vu ce matin : l'ordre des stages, le fail fast, un artefact promu, la sécurité comme un job de plus ? Pas grand-chose. On reprend donc la main, palier par palier, jusqu'à ce que cette pipeline tienne comme une vraie pipeline d'équipe.

Le projet

À la fin des trois paliers, le repo ClickFast contient :

- une pipeline écrite entièrement à la main, avec ses stages dans le bon ordre et ses dépendances explicites entre jobs,
- une image publiée sur Docker Hub, taguée avec le sha du commit, jamais reconstruite entre deux environnements,
- des jobs de sécurité (dépendances, secrets, image, inventaire) qui bloquent la pipeline au-delà d'un certain seuil de gravité,
- un résumé de sécurité lisible d'un coup d'œil dans l'onglet Actions,
- un environnement de production protégé : personne ne publie sur Docker Hub sans qu'un humain ait cliqué sur "approve",
- deux workflows distincts, l'un qui vérifie, l'autre qui publie,
- une entrée de Journal de bord qui prouve qu'on a vu la pipeline passer au rouge et qu'on a su la remettre au vert.

sha ? L'empreinte unique générée automatiquement pour chaque commit, quelques caractères qui identifient exactement ce commit et aucun autre. Un tag d'image basé dessus est imbattable en précision : on sait toujours, sans ambiguïté, quel code tourne derrière quelle image.

Un tableau de bord, ouvert dans le Journal de bord dès le palier 1, garde la trace de tout ça au fil des paliers : durée de chaque run, taille de l'image, nombre de composants dans l'inventaire de sécurité. On y revient à chaque palier, jamais en une seule fois à la fin.

Palier 1 : on reprend la main sur la pipeline

Avant de coder : trois paliers, dix phases, et un principe qui les traverse toutes. On commit une fois par phase terminée, jamais un `git add .` qui mélange tout, et chaque message dit ce que la phase a changé. Le jour où la pipeline casse, c'est ce fil de commits qui permet de retrouver quand exactement.

Phase 1 : des stages qu'on a écrits soi-même

Le workflow généré par GitHub fait tout dans un seul job. On le remplace par une séquence à soi : un stage `lint`, qui n'existait pas du tout dans ClickFast, puis un stage `test` qui reprend les tests Jest déjà écrits, avec l'un qui bloque l'autre s'il casse.

ESLint ? On l'a croisé au chapitre 6 comme exemple de linter. Ici, c'est la première fois qu'on l'installe pour de vrai sur ClickFast, avec sa propre configuration.

Ce qu'il faut produire :

- un fichier de config ESLint à la racine du projet, et un script `npm run lint` dans `package.json`
- un workflow avec deux jobs, `lint` puis `test`, reliés par `needs:` pour que `test` attende `lint`
- les deux jobs tournent sur chaque `push` et chaque `pull_request`, comme dans le Cas 1 du chapitre 7

Devrait afficher : dans l'onglet Actions, deux jobs distincts pour un même run, `lint` toujours terminé avant que `test` ne démarre, visible dans l'ordre d'apparition des logs.

Checkpoint qualité :

- cas normal : un push sur du code propre passe les deux jobs au vert
- cas limite : un fichier avec une erreur de style connue (point-virgule oublié, variable jamais utilisée) fait échouer `lint` et `test` ne se lance jamais
- cas adverse : on renomme sa branche de travail sans toucher au fichier `on:` du workflow. **Rien ne se déclenche du tout, ce qui est le piège le plus fréquent et le plus silencieux de toute la CI**

Phase 2 : un artefact publié, pas reconstruit

Jusqu'ici l'image Docker de ClickFast reste sur votre machine. On la publie pour de vrai sur Docker Hub, avec un tag qui n'est ni `latest` ni un numéro inventé à la main, mais le sha du commit, exactement comme dans la section "Un artefact construit une fois, promu d'environnement en environnement" du chapitre 5. Le geste revient identique sur GitLab-CI dans quelques jours, pour la vraie Todo API : plus vous le solidifiez ici, moins il vous coûtera là-bas.

Rappel notation : chaque secret utilisé dans un workflow (identifiant Docker Hub, jeton) se configure dans les réglages du repo, jamais écrit en clair dans le fichier YAML. **Un commit qui contient un mot de passe en dur, c'est un secret mort dès qu'il est poussé, même supprimé au commit suivant.**

Ce qu'il faut produire :

- un job `build-and-push`, qui dépend de `test`, et qui ne se déclenche que sur `main` (pas sur une pull request : on ne publie jamais du travail en chantier)
- l'authentification passe par l'action [docker/login-action](#) : elle connecte le runner à Docker Hub avec les identifiants stockés en secrets
- la publication passe par [docker/build-push-action](#) : elle build puis pousse l'image en une seule étape

- le tag de l'image doit être `{{ github.sha }}`, la variable qui contient le sha du commit courant

Le test qui compte : deux push successifs sur `main` doivent produire deux images différentes sur Docker Hub, visibles chacune avec son propre tag. Si les deux runs produisent le même tag, quelque chose n'a pas été branché sur `github.sha`.

Trois scénarios à passer :

- un push sur `main` avec des tests verts publie bien une nouvelle image taguée
- une pull request qui build et teste mais ne publie rien, même si tout est vert
- un secret Docker Hub volontairement mal orthographié fait échouer le job `build-and-push`, et seulement celui-là : `lint` et `test` restent verts, la preuve que l'échec est bien localisé

Phase 3 : mesurer avant d'optimiser

Avant de lancer : on ouvre, dans le Journal de bord, le tableau de bord annoncé dans "Le projet". Trois colonnes pour commencer : durée totale du run, durée du job `test`, taille de l'image publiée (`docker images` en local, ou la taille affichée sur Docker Hub). On le remplit maintenant, avant de toucher au cache, pour avoir une valeur de référence.

Ensuite, `cache: 'npm'` sur `actions/setup-node` : exactement le réglage du Cas 1 au chapitre 7, celui qui faisait passer l'installation de 30 à 8 secondes.

Ce qui doit être vrai à la fin :

- une ligne "avant cache" et une ligne "après cache" dans le tableau de bord, avec un écart mesuré et pas estimé
- si l'écart est nul ou négatif, le cache est mal configuré (mauvaise clé, mauvais chemin) avant d'être inutile
- une branche qui réutilise, sans rapport avec elle, la clé de cache d'une autre branche ne doit jamais faire apparaître de dépendances absentes de son propre `package.json` : un cache partagé accélère l'installation, il ne doit jamais remplacer ce qu'il y a vraiment dedans
- un commit qui touche un seul fichier source, sans toucher aux dépendances, doit profiter pleinement du cache

On a maintenant une pipeline qui nous appartient, qui publie un vrai artefact et qu'on sait chronométrer. Il lui manque une chose que le fichier généré par GitHub n'a jamais eue : elle ne vérifie rien côté sécurité. C'est le sujet du palier suivant, et il tombe au bon moment : le chapitre 8 vient d'en donner tout le vocabulaire.

Palier 2 : la sécurité entre dans la pipeline

Trois familles de scanners ont été vues au chapitre 8 : le SCA avec `npm audit`, le secret scanning avec [gitleaks](#), le container scanning avec [Trivy](#). ClickFast n'en a encore branché aucun. On corrige ça scanner par scanner, chacun avec sa question, avant de les faire parler ensemble à la fin.

Phase 4 : les dépendances et les secrets

Côté commits : un job de sécurité rouge n'est pas un job cassé, c'est un job qui a fait exactement son travail. On commit la configuration même si elle fait apparaître une alerte : c'est la trace du diagnostic, pas une honte à cacher.

Le job vu au chapitre 8 tourne sur un projet générique. Ici, on le branche sur ClickFast lui-même, avec ses propres dépendances et son propre historique Git.

CVE ? Common Vulnerabilities and Exposures : l'identifiant standard attribué à chaque faille de sécurité déjà publiée et documentée. Quand `npm audit` ou Trivy remontent une CVE, c'est une vulnérabilité connue et référencée, pas une supposition.

Ce qu'il faut produire :

- un job `security-deps`, qui tourne en parallèle de `lint` et `test` (pas après : rien n'empêche de vérifier les dépendances pendant que les tests tournent)
- `npm audit --audit-level=high`, avec le même seuil que celui vu en cours : on bloque sur `high` et `critical`, on laisse passer le reste
- gitleaks branché via `gitleaks/gitleaks-action@v2`, qui relit tout l'historique du dépôt

Qu'est-ce qui casse si...

- une dépendance volontairement vieillie dans `package.json` (une version d'il y a plusieurs années) a de bonnes chances de posséder une CVE connue : le job doit la trouver
- un faux secret collé dans un commit de test, puis supprimé au commit suivant, doit quand même être détecté par gitleaks : c'est précisément ce qu'il vérifie
- un job de sécurité qui reste vert alors qu'on vient d'y injecter une vulnérabilité connue n'a rien vérifié du tout : direction la configuration du seuil

Phase 5 : l'image elle-même

`npm audit` regarde le code de l'application. Il ne voit rien de ce qui vit dans l'image Docker : la version de nginx, les paquets système embarqués. C'est exactement la question que pose le container scanning.

Rappel notation : Trivy scanne une image déjà construite, donc ce job dépend du job `build-and-push` de la phase 2. Il n'a aucune raison de tourner sur une pull request qui n'a rien publié.

Ce qu'il faut produire :

- un job `security-image`, après `build-and-push`, qui scanne le tag qui vient d'être publié
- un seuil de blocage sur `HIGH` et `CRITICAL`, cohérent avec celui choisi en phase 4

Preuve de bon fonctionnement :

- l'image `nginx:alpine` actuelle passe (ou remonte une liste courte, alpine est réputé léger)
- en changeant volontairement la ligne `FROM` pour une version de nginx vieille de plusieurs années, le job doit remonter des CVE connues et échouer
- un tag qui n'existe pas encore sur Docker Hub au moment du scan (le job `security-image` lancé avant que `build-and-push` n'ait terminé) doit faire échouer proprement le job, jamais planter en silence

Et côté tableau de bord : une colonne de plus, le nombre de vulnérabilités `HIGH`/`CRITICAL` trouvées à ce commit.

Phase 6 : savoir ce qu'on livre, pas seulement si c'est sûr

Les deux scanners précédents répondent "y a-t-il un problème connu ?". Une question voisine, et tout aussi utile le jour où une faille majeure est annoncée dans une bibliothèque très répandue : *qu'est-ce qu'il y a exactement dans ce qu'on publie ?*

SBOM ? On l'a défini au chapitre 8 : la liste complète, lisible par une machine, de tout ce que contient un livrable. [Syft](#) en génère un à partir d'une image Docker.

Ce qu'il faut produire :

- un job qui génère un SBOM de l'image ClickFast avec Syft, au format [CycloneDX](#) (déjà cité au chapitre 8)
- le fichier résultant exporté comme artefact GitHub Actions, téléchargeable depuis l'onglet Actions, avec `actions/upload-artifact@v4` vu au chapitre 7

Ce qui doit être vrai à la fin :

- télécharger l'artefact depuis un run récent doit donner un fichier lisible, avec au moins le nom et la version de chaque composant
- si le fichier est vide ou absent, le job a probablement tourné avant que l'image ne soit poussée : vérifier l'ordre des `needs :`
- un numéro de version délibérément inventé dans `package.json` doit quand même apparaître tel quel dans le SBOM : Syft recense ce qui est déclaré, il ne vérifie pas si c'est vrai
- deux commits qui changent une dépendance doivent produire deux SBOM différents, comparables l'un à l'autre

Phase 7 : un seul endroit où lire l'état de sécurité

On a maintenant quatre sources d'information séparées : l'audit de dépendances, gitleaks, Trivy, le SBOM. Quatre jobs, quatre onglets à ouvrir pour savoir si on peut publier tranquille. Ça ne passe pas à l'échelle : sur un vrai projet, personne ne va lire quatre logs différents à chaque pull request.

`$GITHUB_STEP_SUMMARY` ? Une variable spéciale de GitHub Actions : tout ce qu'on y écrit devient un résumé Markdown affiché directement dans l'interface du run, sans avoir à ouvrir les logs. Doc : [workflow commands, section "adding a job summary"](#).

Cette phase n'a pas de solution unique, et c'est voulu. Le résumé agrège les quatre jobs précédents : nombre d'alertes de dépendances, nombre de secrets trouvés, nombre de CVE sur l'image, nombre de composants dans le SBOM. Il s'écrit dans `$GITHUB_STEP_SUMMARY`, depuis un job final qui dépend des quatre autres. La forme du tableau, le niveau de détail, l'ajout d'une ligne "verdict global" : c'est un chantier qui reste ouvert, on peut toujours l'enrichir d'une colonne de plus, d'un lien de plus vers chaque rapport détaillé.

Ce que le résumé doit permettre :

- répondre en une lecture à *peut-on publier cette version en confiance ?*, sans ouvrir un seul log détaillé
- rester correct même quand un des quatre jobs a été sauté (par exemple sur une pull request qui n'a pas encore d'image à scanner)
- rester correct même quand un des quatre jobs plante avant d'écrire son résultat (erreur réseau, timeout) : le résumé ne doit ni planter à son tour, ni afficher une valeur inventée
- survivre à l'ajout d'un cinquième scanner plus tard, sans qu'il faille tout réécrire

La pipeline sait maintenant construire, publier et vérifier. Il lui manque la dernière pièce, et c'est la plus politique des trois paliers : qui a le droit de dire "cette version part vraiment en public", et est-ce que ça doit être automatique.

Palier 3 : le geste qui compte, et savoir l'arrêter

Le chapitre 3 distinguait Continuous Delivery (un humain valide le "go" final) et Continuous Deployment (tout est automatique, y compris le "go"). Jusqu'ici, notre pipeline fait du Deployment complet : dès que `main` est vert, l'image part sur Docker Hub sans que personne ne la regarde. On arrête la théorie et on pose un vrai bouton à cliquer, pas juste une définition apprise par cœur.

Phase 8 : un humain avant la publication

GitHub Environments ? Un environnement nommé (`production`, `staging`...) qu'on associe à un job. On peut lui attacher des règles de protection, dont la plus utile ici : exiger qu'une personne précise clique sur "approve" avant que le job ne démarre. Doc : [using environments for deployment](#).

Trace attendue : dans le Journal de bord, une capture ou une description du moment où le job attend une validation, avec le nom de la personne qui a approuvé et l'heure.

Ce qu'il faut produire :

- un environnement `production` créé dans les réglages du repo, avec un reviewer obligatoire (vous-même, ou un camarade de binôme)
- le job `build-and-push` de la phase 2 rattaché à cet environnement via `environment: production`
- le run reste "en attente" dans l'onglet Actions tant que personne n'a validé, exactement le "go" manuel du chapitre 3

Les scénarios à passer :

- un push sur `main` construit l'image et s'arrête juste avant la publication, en attente de validation
- une fois validé, la publication reprend et se termine normalement
- si personne ne valide dans un délai raisonnable, le run reste en attente indéfiniment : c'est le comportement voulu, pas un bug
- une personne qui n'est pas listée comme reviewer sur l'environnement `production` ne voit aucun bouton "approve" en ouvrant le run : à vérifier avec un second compte GitHub, par exemple celui d'un camarade de binôme

Phase 9 : deux workflows, deux intentions

Un seul fichier fait tout depuis la phase 1 : vérifier et publier. Sur un vrai projet, une pull request ne doit jamais pouvoir déclencher une publication, même par accident. On sépare donc en deux fichiers distincts dans `.github/workflows/`, avec la même logique de déclencheurs que le Cas 1 du chapitre 7 : un `on: pull_request` qui vérifie seulement, un `on: push` sur `main` qui publie.

Ce qu'il faut produire :

- `verify.yml` : lint, tests, scans de sécurité, déclenché sur toute pull request
- `release.yml` : la suite complète jusqu'à la publication protégée par l'environnement, déclenché seulement sur un push vers `main`

- aucune duplication totale entre les deux : les jobs communs peuvent être répétés dans les deux fichiers (GitHub Actions ne partage pas facilement du code entre workflows), mais leurs déclencheurs restent strictement séparés

Checkpoint qualité :

- une pull request lance `verify.yml` et jamais `release.yml`
- un push direct sur `main` (après fusion de la pull request) lance `release.yml`
- une branche nommée `master` au lieu de `main` ne déclenche rien du tout : c'est l'erreur la plus fréquente et la plus silencieuse de toute la CI, déjà annoncée en phase 1, et qui revient ici sous une forme différente parce que deux fichiers valent deux occasions de se tromper

Phase 10 : casser main, et savoir s'en remettre

Le chapitre 5 l'affirmait sans le montrer : **"une pipeline rouge sur `main` n'est pas un problème personnel, c'est un arrêt d'usine"**. On va le vivre pour de vrai, sur son propre repo. Une seule fois suffit pour que le réflexe reste.

Avant de coder : ce commit est volontaire et va casser quelque chose. On le fait sur une branche, jamais directement sur `main`, exactement comme le workflow `verify.yml` de la phase 9 est fait pour l'attraper avant la fusion.

Le protocole :

1. sur une branche, on casse volontairement un test Jest existant (on inverse une assertion, par exemple)
2. on ouvre une pull request : `verify.yml` doit tourner et échouer
3. malgré tout, on fusionne la pull request sur `main` (l'occasion de voir ce qui se passe quand une branche protégée n'est pas encore configurée pour bloquer une fusion sur CI rouge)
4. on observe `release.yml` échouer à son tour sur `main`
5. on écrit dans le Journal de bord ce qui s'est passé à l'étape 3, et la correction possible : rendre le check `verify.yml` obligatoire avant fusion, dans les réglages de la branche `main`
6. on corrige le test, on repousse, on regarde toute la chaîne redevenir verte

Ce qui doit être vrai à la fin :

- le tableau de bord du Journal de bord gagne une dernière ligne : le temps écoulé entre le commit qui casse et le commit qui répare, le propre "temps de rétablissement" de la métrique DORA vue au chapitre 3
- l'image publiée sur Docker Hub avant la casse reste disponible et fonctionnelle : personne n'a touché à l'ancien tag, seul un nouveau tag est arrivé une fois corrigé
- une deuxième personne du groupe qui relit le Journal de bord doit comprendre, sans avoir vu l'incident en direct, ce qui s'est passé et pourquoi

Dix phases plus tard, ClickFast n'a plus rien d'un projet d'échauffement : sa pipeline s'écrit à la main, publie un artefact traçable, se protège elle-même côté sécurité, et demande un humain avant de rendre quoi que ce soit public. La vraie Todo API, sur GitLab-CI, reprend exactement les mêmes idées : ce n'est plus la logique qui sera nouvelle, seulement la syntaxe.